What Matters in Deep RL

On-Policy Algorithms, Off-Policy Algorithms, Implementation Details and Sample Efficiency

D. Sorokin

AIMasters RL Course · Lecture 12

Paper List

#	Paper	Venue
1	Deep Reinforcement Learning that Matters - Handerson et al.	AAAI 2018
2	Implementation Matters (PPO/TRPO) — Engstrom et al.	ICLR 2020
3	What Matters for On-Policy AC? — Andrychowicz et al.	ICLR 2021
4	37 Details of PPO — Huang et al.	ICLR Blog 2022
5	TD3 — Fujimoto, van Hoof & Meger	ICML 2018
6	Soft Actor-Critic — Haarnoja et al.	ICML 2018
7	SAC Algorithms & Applications — Haarnoja et al.	arXiv 2019
8	TQC — Kuznetsov et al.	ICML 2020
9	Fundamentals of Experience Replay — Fedus et al.	ICML 2020
10	REDQ — Chen et al.	ICLR 2021
11	DroQ — Hiraoka et al.	ICLR 2022
12	Primacy Bias — Nikishin et al.	ICML 2022
13	Capacity Loss — Lyle, Rowland & Dabney	ICLR 2022
14	SR-SAC / Replay Ratio Barrier — D'Oro et al.	ICLR 2023

Roadmap

1. **On-Policy Methods** — what *actually* makes PPO work
   Papers 1–4
2. **Off-Policy Methods**
   - 2.1 Foundations
   - 2.2 Overestimation bias and sampling strategy (TD3, SAC, TQC)
     Papers 5–8
   - 2.3 Replay Ratio & High-UTD — Papers 8–10
   - 2.4 **Plasticity & Capacity Loss** — Papers 11–12
   - 2.5 **SR-SAC** — Paper 14

On-Policy Methods

Paper 1 – The Reproducibility Crisis in Deep RL

Henderson et al. (2018) show:

- Large variance across random seeds alone
- Hyperparameter choices interact with environment in unpredictable ways

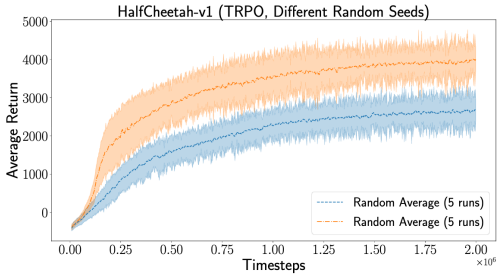


Figure 5: TRPO on HalfCheetah-v1 using the same hyperparameter configurations averaged over two sets of 5 different random seeds each. The average 2-sample t-test across entire training distribution resulted in $t = -9.0916$, $p = 0.0016$.

PPO Algorithm

Init policy π_θ, value fn V_ϕ
for each iteration **do**
  $\theta_{\text{old}} \leftarrow \theta$ *(save current policy)*
  Collect T steps with $\pi_{\theta_{\text{old}}}$; store in $\mathcal{D}$
  Compute advantages $\hat{A}_t$ via GAE
  for K epochs **do**
    for each minibatch $\mathcal{B} \subset \mathcal{D}$ **do**
      $\rho_t(\theta) \leftarrow \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
      $L^{\text{CLIP}} \leftarrow \mathbb{E}\big[\min\big(\rho_t\hat{A}_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\big]$
      $L^{\text{VF}} \leftarrow \mathbb{E}\big[\big(V_\phi(s_t) - V_t^{\text{targ}}\big)^2\big]$
      Update θ, ϕ: maximise $L^{\text{CLIP}} - c_1 L^{\text{VF}} + c_2\mathcal{H}[\pi_\theta]$
    end for
  end for
end for

Key design choices:

- ε — clip range (typically 0.1–0.2)
- K — number of epochs per rollout
- λ — GAE parameter (≈ 0.9)
- c_1, c_2 — VF and entropy coefficients
- Observation normalisation
- LR annealing schedule

The performance comes from the choices above.

Paper 2 — Implementation Matters in Deep Policy Gradients

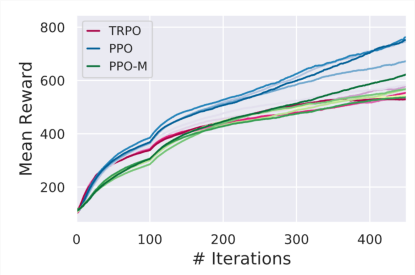
Engstrom et al. (ICLR 2020)

arXiv:2005.12729

Setup: Re-implement PPO and TRPO in one codebase; ablate code-level tricks one by one.

9 code-level tricks studied:

1. Value function clipping (PPO-style clip on V_θ)
2. Reward scaling (divide by std of discounted return)
3. Orthogonal init + per-layer gain scaling
4. Adam LR annealing
5. Reward clipping (e.g. $[-5, 5]$)
6. Observation normalization (mean-zero, unit-var)
7. Observation clipping (e.g. $[-10, 10]$)
8. Tanh activations
9. Global gradient norm clipping (≤ 0.5)



Code-level optimizations alone account for most of PPO's advantage over TRPO.

Paper 3 — What Matters for On-Policy Actor-Critic Methods?

Andrychowicz et al. (ICLR 2021)

arXiv:2006.05990

Scale: 250 000+ agents across 50+ binary/continuous choices in a unified PPO framework.

Category	Top finding
Policy loss	PPO, ε=0.25; value clipping doesn't help
Architecture	Tanh; separate policy/value nets; 2 layers
Init	Last policy layer $\times$100 smaller
Normalisation	Obs. norm. crucial; value norm. env-dep.
Advantage	GAE λ=0.9; no Huber loss
Training	Multiple passes; shuffle transitions; recompute $\hat{A}$
Optimiser	Adam or RMSProp similar; LR 3×10^{-4}
Regularisation	Entropy bonus & KL don't help — PPO clip + good init suffice

Paper 4 — The 37 Implementation Details of PPO

Huang et al. (ICLR Blog Track 2022)

iclr-blog-track.github.io

Annotated walkthrough of a complete PPO (CleanRL) — every non-obvious decision numbered and explained.

Representative details:

- Value-loss clipping mirrors policy clip (adds little empirically)
- Invalid action masking in discrete spaces
- Clip-range *and* LR anneal together, not independently

Use CleanRL/SB3 for reproducible PPO reference

Detail #1: Vectorised environments

Detail #4: Orthogonal init, $\sigma = \sqrt{2}$

Detail #9: Obs normalisation (running)

Detail #12: Adv. norm. per minibatch

Detail #17: Value loss clipping

. . . 31 more details . . .

On-Policy Summary

1. The **algorithm label** (PPO vs. TRPO) is less informative than the **implementation**.
2. **Observation normalisation** is the single highest-leverage switch.
3. Good performance is a **conjunction** of > 10 individually modest choices.
4. Use SB3/CleanRL for reproducible PPO.

On-policy methods are stable but sample-hungry.
Off-policy methods reuse past experience. Do they have failure modes?

Off-Policy Foundations: DDPG, TD3, SAC and TQC

DDPG Algorithm

Init μ_θ, Q_ϕ; target nets $\mu_{\theta'}$, $Q_{\phi'}$; buffer $\mathcal{D}$
for each environment step **do**
    $a_t = \mu_\theta(s_t) + \mathcal{N}_t$ *(exploration noise)*
    Execute a_t; store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
    Sample minibatch $\mathcal{B}$ from $\mathcal{D}$
    $y_i = r_i + \gamma\, Q_{\phi'}(s_{i+1},\, \mu_{\theta'}(s_{i+1}))$
    Update critic: $\min_\phi \mathbb{E}_{\mathcal{B}}\big[(Q_\phi(s_i, a_i) - y_i)^2\big]$
    Update actor: $\max_\theta \mathbb{E}_{\mathcal{B}}\big[Q_\phi(s_i, \mu_\theta(s_i))\big]$
    Soft-update targets: $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$
end for

Key components:

- Deterministic policy μ_θ
- Off-policy via replay buffer $\mathcal{D}$
- Target networks (soft update $\tau \ll 1$)
- Exploration by injecting $\mathcal{N}_t$

Failure modes

1. Q overestimation
2. Brittle to hyperparams

Paper 5 — TD3: Twin Delayed Deep Deterministic Policy Gradient

Fujimoto, van Hoof & Meger (ICML 2018)

arXiv:1802.09477

Root cause: Deterministic policy exploits Q-function errors ⇒ overestimation compounds.

1. **Clipped double-Q** (same insight as SAC):

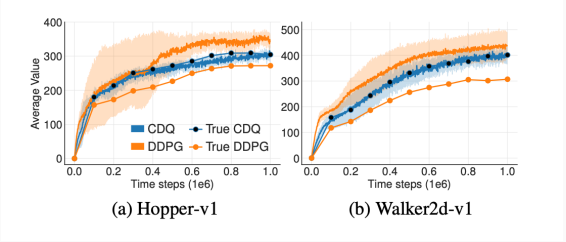
$$y = r + \gamma \min_{i=1,2} Q_i\big(s', \pi(s') + \epsilon\big)$$

2. **Delayed policy updates:** Update actor every $d = 2$ critic steps — critic is more accurate first.

3. **Target policy smoothing:**

$$\epsilon \sim \mathrm{clip}(\mathcal{N}(0, \sigma),\ -c,\ c)$$

Prevents actor from exploiting sharp Q peaks.



(a) Hopper-v1 (b) Walker2d-v1

Paper 6 — Soft Actor-Critic (SAC)

Haarnoja et al. (ICML 2018)
arXiv:1801.01290

1. **Maximum entropy objective**

$$J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha\, \mathcal{H}(\pi(\cdot|s_t))]$$

2. **Clipped double-Q:** Two critics Q_1, Q_2; take min for targets $\Rightarrow$ conservative value estimates.
3. **Reparameterisation trick:** $a = f_\phi(\epsilon, s)$, $\epsilon \sim \mathcal{N}(0, I)$ — enables direct policy gradient.

Soft Bellman / policy loss:

$$V(s) = \mathbb{E}_{a\sim\pi}\big[Q(s,a) - \alpha \log \pi(a|s)\big]$$

$$J(\phi) = \mathbb{E}_{s,\epsilon}\big[\alpha \log \pi_\phi(a|s) - Q(s,a)\big]$$

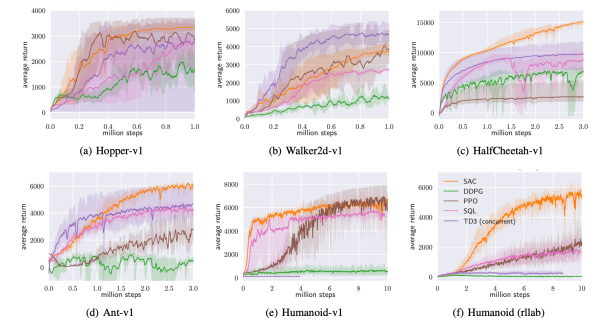


Figure 1. Training curves on continuous control benchmarks. Soft actor-critic (yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

Paper 7 — SAC v2: Automatic Temperature Tuning

Haarnoja et al. (2019)

arXiv:1812.05905

Problem with SAC v1: Temperature α is a manual hyperparameter. Wrong $\alpha \Rightarrow$ policy collapses or fails to explore.

Fix: constrained optimisation

$$\max_{\pi} \mathbb{E}\Big[\sum_t r_t\Big] \text{ s.t. } \mathbb{E}\big[-\log \pi(a_t|s_t)\big] \geq \mathcal{H}_{\text{target}}$$

Typical target: $\mathcal{H}_{\text{target}} = -|\mathcal{A}|$

Dual gradient descent automatically adjusts α:

$$\alpha^* = \arg\min_{\alpha} \; \mathbb{E}_{a\sim\pi}\big[-\alpha \log \pi(a|s) - \alpha \mathcal{H}_{\text{target}}\big]$$

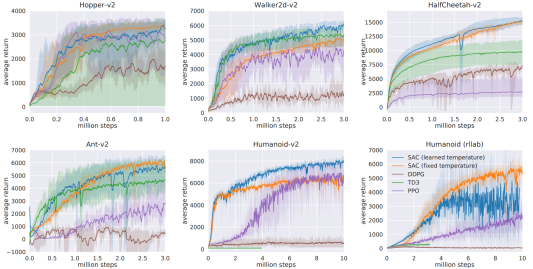


Figure 1: Training curves on continuous control benchmarks. Soft actor-critic (blue and yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

Paper 8 — TQC: Truncated Quantile Critics

Kuznetsov et al. (ICML 2020)

arXiv:2005.04269

Idea: Clipped double-Q is a crude fix — just take the minimum of two scalars. Can we control overestimation more precisely?

Distributional critic: Each of N critics outputs K quantile values $\hat{z}_{ik}(s, a)$ instead of a single $Q(s, a)$.

Truncated target: Gather all $N \times K$ quantile atoms from target critics, drop the top d atoms, use the rest as regression targets.

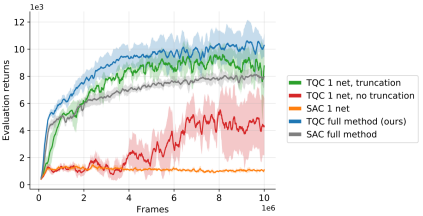


Figure 1. Evaluation on the Humanoid environment. Results are averaged over 4 seeds, ± std is shaded.

d controls bias–variance tradeoff: $d = 0 \Rightarrow$ no truncation, $d = NK \Rightarrow$ very conservative

Result: d is a single, interpretable hyperparameter. TQC outperforms SAC and TD3 on all standard MuJoCo benchmarks.

SAC $\subset$ TQC

With $N = 2$, $K = 1$, $d = 1$:

TQC recovers clipped double-Q.

TQC strictly generalises SAC/TD3.

Off-Policy Summary

1. **Clipped double-Q** (SAC, TD3) is the key fix for overestimation.
2. **Entropy regularisation** (SAC) improves exploration and removes manual schedules.
3. **Auto-tuned temperature** (SAC v2) removes the last per-task hyperparameter.
4. **TQC** replaces scalar Q with quantile distributions — truncation gives a principled, tunable knob over the bias–variance tradeoff.

All of the above use replay ratio $= 1$.
Can we do better?

Experience Replay and the Replay Ratio

Paper 9 — Revisiting Fundamentals of Experience Replay

Fedus et al. (ICML 2020)

arXiv:2007.06700

Prior work conflated two independent variables:

- **Replay capacity** — how many transitions stored
- **Replay ratio (UTD)** — gradient updates per env step

Findings:

- Larger buffers help — but diminishing returns; staleness hurts
- Replay ratio is the dominant lever for sample efficiency
- Coins the standard term **"replay ratio"** (now universal)

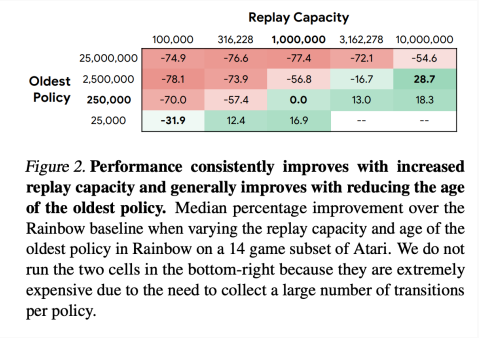


Figure 2. **Performance consistently improves with increased replay capacity and generally improves with reducing the age of the oldest policy.** Median percentage improvement over the Rainbow baseline when varying the replay capacity and age of the oldest policy in Rainbow on a 14 game subset of Atari. We do not run the two cells in the bottom-right because they are extremely expensive due to the need to collect a large number of transitions per policy.

Paper 10 — REDQ: High UTD with Randomised Ensembles

Chen et al. (ICLR 2021)

arXiv:2101.05982

Goal: Match model-based sample efficiency *without* a world model.

Problem: Naively set UTD $= 20$ with SAC $\Rightarrow$ Q-values explode.

REDQ fix — three components:

1. High UTD: $G = 20$ gradient steps per env step
2. Large ensemble: $N = 10$ Q-networks
3. Subsample $M = 2$ networks per update for target:
   $y = r + \gamma \min_{i \in \mathcal{M}} Q_i(s', a')$

Result: variance reduction at fraction of full-ensemble cost.

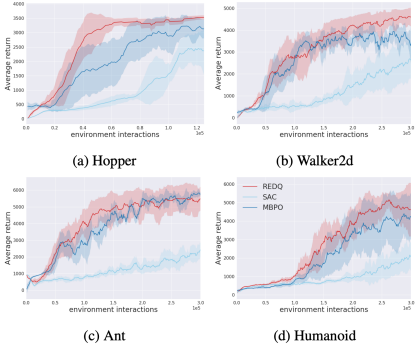


Figure 1: REDQ compared to MBPO and SAC. Both REDQ and MBPO use $G = 20$.

$N = 10$ Q-networks is expensive.

Can we get the same with two networks?

Paper 11 — DroQ: Doubly Efficient High-UTD Training

Hiraoka et al. (ICLR 2022)

arXiv:2110.02034

DroQ = SAC + 3 additions:

1. **Dropout** on Q-networks — implicit ensemble, variance reduction
2. **Layer normalisation** before each hidden layer
3. High **UTD = 20**

Ablation finding:

- Drop LN ⇒ training still diverges at UTD=20
- Drop dropout ⇒ modest performance drop
- Layer normalisation is the crucial stabiliser

LN prevents feature scale explosion.

Table 1: Process times (in msec) per overall loop (e.g., lines 3–10 in Algorithm 2). Process times per Q-functions update (e.g., lines 4–9 in Algorithm 2) are shown in parentheses.

	Hopper-v2	Walker2d-v2	Ant-v2	Humanoid-v2
SAC	910 (870)	870 (835)	888 (848)	893 (854)
REDQ	2328 (2269)	2339 (2283)	2336 (2277)	2400 (2340)
DUVN	664 (636)	762 (731)	733 (700)	692 (660)
DroQ	948 (905)	933 (892)	954 (913)	989 (946)

The two efficiencies

Sample-efficient: matches REDQ

Compute-efficient: matches SAC

"Doubly efficient"

Replay Ratio Summary

Section 3 takeaways

1. **Replay ratio** (UTD) is the primary lever for sample efficiency.
2. Simply increasing UTD with vanilla SAC fails — value divergence.
3. **REDQ** fixes this with randomised ensembles ($N = 10$, $G = 20$), matching model-based efficiency.
4. **DroQ** achieves the same at SAC compute cost via dropout + layer normalisation.
5. Layer norm emerges as a mysterious stabiliser — *why*?

The next two papers give the mechanistic explanation — and why layer norm + resets fix it.

Plasticity, Capacity Loss, and the High-UTD Failure Mode

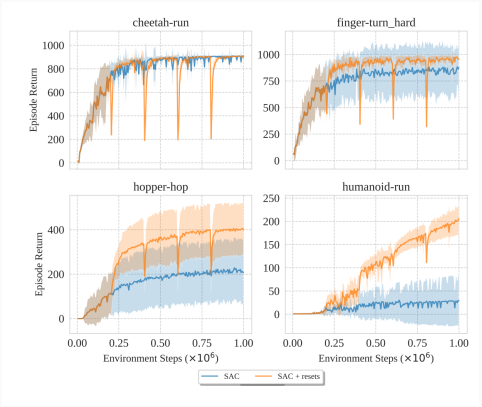
Paper 12 — The Primacy Bias in Deep RL

Nikishin et al. (ICML 2022)

arXiv:2205.07802

Phenomenon: Under high replay ratios, networks overfit to early transitions — even after those have left the buffer.

- Early data shapes the loss landscape; weights get "locked in"
- Standard replay adds data but doesn't undo biased weights
- **Fix:** every K steps, reinitialise network weights and retrain on the *current* buffer
- The knowledge is in the **buffer**, not the **network**



Discarding learned weights periodically improves final performance.

Paper 14 — SR-SAC: Breaking the Replay Ratio Barrier

D'Oro et al. (ICLR 2023)

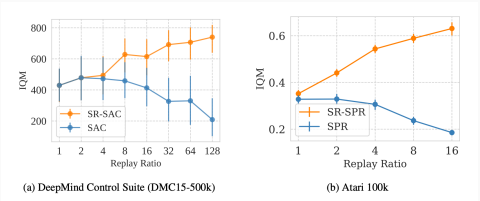
openreview.net/forum?id=OpC-9aBBVJe

SR-SAC = SAC + layer norm + periodic resets + UTD = 128

Combines Papers 12 + 13:

- Resets fix primacy bias (Paper 12 — Nikishin et al.)
- Layer norm maintains feature diversity (Paper 13 — Lyle et al.)
- Together: stable training at UTD up to 128

"The barrier": Prior to this, UTD ≈ 20 was an implicit ceiling — SR-SAC shows it was an artifact of plasticity failure.
Result: Nearly $2\times$ performance over SAC for identical env steps.



Open questions

- Optimal reset frequency?
- Warm vs. cold restarts?
- Sparse rewards, pixel obs?
- Scales to LLM-based RL?

Synthesis

The Complete Story

Problem	Fix	Papers
PPO gains = implementation, not algorithm	Careful ablations, code standards	1–4
Q-overestimation in off-policy AC	Double-Q + entropy + delayed updates	5, 6, 7
Scalar Q too coarse for overestimation	Distributional Q + quantile truncation	8 (TQC)
Sample inefficiency (UTD = 1)	Replay ratio as the key lever	9
High UTD diverges	Ensembles / dropout + layer norm	10, 11
Primacy bias / capacity loss	Periodic network resets	12, 13
Resets + layer norm unlock UTD $\gg$ 20	SR-SAC	14

Three Key Takeaways

1. **Implementation is not a second-class concern.**
   The gap between a paper's claim and a reproducible result lives in implementation details. When you can't reproduce a result, look at the code first.

2. **Sample efficiency $\approx$ replay ratio — but only if you solve plasticity.**
   Layer norm + periodic resets together unlock a new performance regime. Neither alone is sufficient; the mechanism is now understood (rank collapse).

3. **Plasticity failure is a fundamental RL pathology.**
   It arises from bootstrapping + non-stationary targets + function approximation. Expect this to be a recurring theme as RL scales.